Индивидуальный проект. Этап №1

Размещение на Github pages заготовки для персонального сайта

Смирнов Артём Сергеевич

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение индивидуального проекта **8**

4.1 Установка программного обеспечения 8
4.2 Создание репозитория на GitHub 9
4.3 Клонирование репозитория 11
4.4 Установка зависимостей проекта 12
4.5 Локальный запуск сайта 12
4.6 Настройка baseURL 13
4.7 Настройка GitHub Pages 14
4.8 Публикация сайта 14
4.9 Проверка результата 16

5 Выводы **17**

Список литературы **18**

Список иллюстраций

4.1 Установка git, golang, nodejs, npm 8
4.2 Проверка версий установленного ПО 9
4.3 Установка Hugo Extended 9
4.4 Создание репозитория по шаблону 10
4.5 Созданный репозиторий на GitHub 11
4.6 Клонирование репозитория 11
4.7 Установка npm-зависимостей 12
4.8 Запуск локального сервера Hugo 12
4.9 Локальный сайт в браузере 13
4.10 Настройка baseURL в config.yaml 14
4.11 Настройка GitHub Pages 14
4.12 Коммит и пуш изменений 15
4.13 Успешный деплой в GitHub Actions 15
4.14 Сайт на публичном адресе 16

Список таблиц

3.1 Используемые технологии и инструменты 7

1 Цель работы

Разместить на Github pages заготовку для персонального сайта научного работника: установить необходимое программное обеспечение, скачать шаблон темы сайта, разместить его на хостинге git и настроить параметры для корректного отображения сайта.

2 Задание

- Установить необходимое программное обеспечение (Git, Go, Node.js, Hugo Extended)
- Скачать шаблон темы сайта Hugo Academic
- Создать репозиторий на GitHub и разместить шаблон
- Установить параметр baseURL для корректной работы сайта
- Разместить заготовку сайта на Github pages

3 Теоретическое введение

Hugo — генератор статических сайтов, написанный на языке Go. Он позволяет создавать быстрые и безопасные веб-сайты без необходимости использования баз данных или серверных скриптов. Hugo использует шаблоны и Markdown-файлы для генерации HTML-страниц.

Hugo Academic Theme (ныне HugoBlox) — популярный шаблон для создания персональных академических сайтов. Он предоставляет готовую структуру для размещения информации о публикациях, проектах, курсах и биографии исследователя.

GitHub Pages — бесплатный хостинг статических сайтов от GitHub. Позволяет публиковать сайты напрямую из репозитория, поддерживает автоматическую сборку через GitHub Actions.

Основные технологии и инструменты представлены в таблице 3.1.

Таблица 3.1: Используемые технологии и инструменты

Инструмент	Описание
Hugo	Генератор статических сайтов
Go	Язык программирования (необходим для Hugo)
Node.js/npm	Среда выполнения JavaScript (для зависимостей темы)
Git	Система контроля версий
GitHub Pages	Хостинг статических сайтов
GitHub Actions	CI/CD для автоматической сборки и деплоя

4 Выполнение индивидуального проекта

4.1 Установка программного обеспечения

Устанавливаю необходимые пакеты через менеджер пакетов dnf(рис. 4.1).

```
sudo dnf install git golang nodejs npm
```

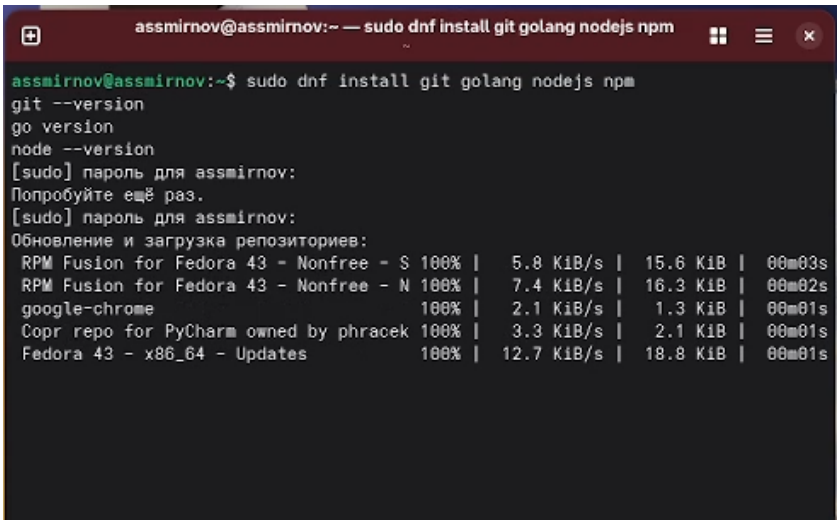

Рис. 4.1: Установка git, golang, nodejs, npm

Проверяю корректность установки(рис. 4.2).

```
git --version
go version
node --version
```

Рис. 4.2: Проверка версий установленного ПО

Устанавливаю Hugo Extended вручную. Скачиваю архив с официального репозитория GitHub, распаковываю его и перемещаю бинарный файл в `/usr/local/bin/`. Проверяю версию — важно, чтобы была версия Extended (рис. 4.3).

```
tar -xzf ~/Загрузки/hugo_extended_*.tar.gz
sudo mv hugo /usr/local/bin/
hugo version
```

Рис. 4.3: Установка Hugo Extended

4.2 Создание репозитория на GitHub

Перехожу на страницу шаблона HugoBlox/hugo-theme-academic-cv на GitHub и нажимаю «Use this template». Создаю новый репозиторий с именем `SmirnovArtemS.github.io` — это важно, так как имя репозитория должно соответствовать формату `USERNAME.github.io` для корректной работы GitHub Pages (рис. 4.4).

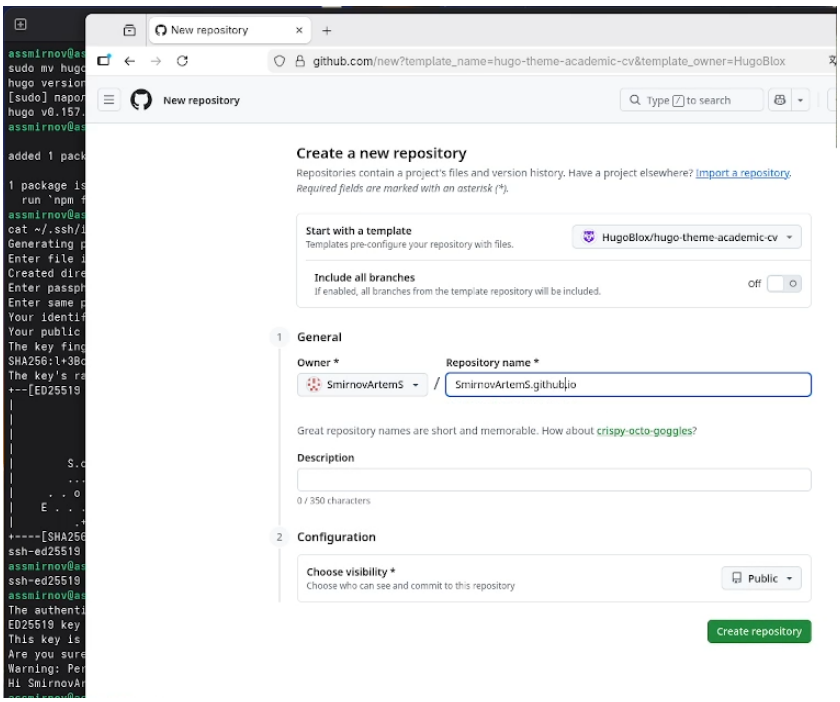

Рис. 4.4: Создание репозитория по шаблону

Репозиторий успешно создан на основе шаблона. Он содержит все необходимые файлы конфигурации и структуру для Hugo-сайта (рис. 4.5).

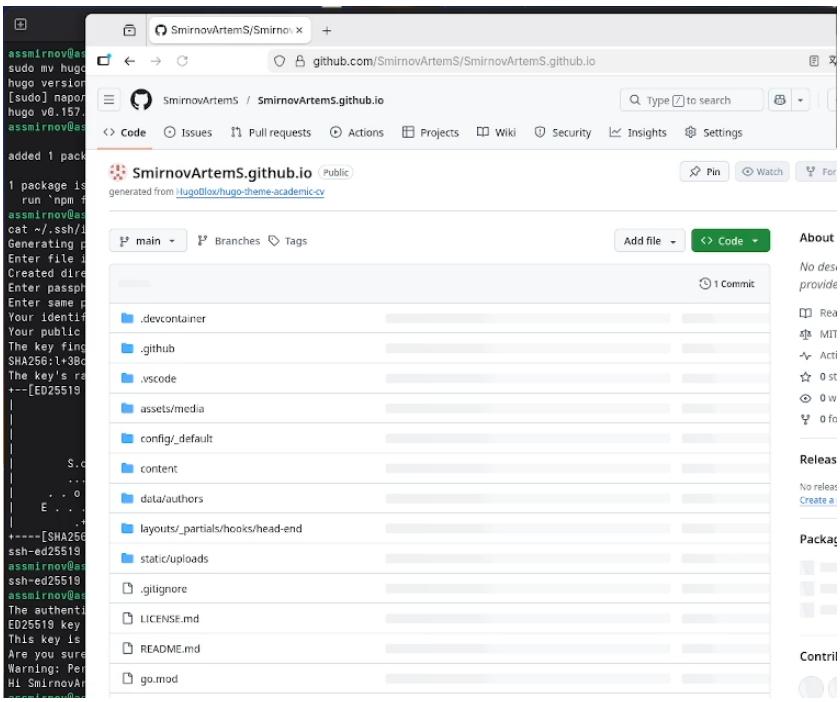

Рис. 4.5: Созданный репозиторий на GitHub

4.3 Клонирование репозитория

Клонирую созданный репозиторий на локальную машину с флагом `--recurse-submodules`(рис. 4.6).

```
git clone --recurse-submodules git@github.com:SmirnovArtemS/SmirnovArtemS.github.io.gi
cd SmirnovArtemS.io
```

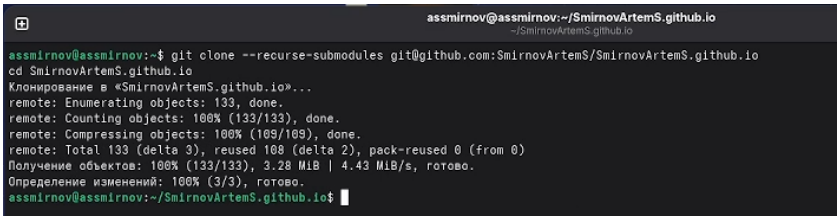

Рис. 4.6: Клонирование репозитория

4.4 Установка зависимостей проекта

Устанавливаю npm-зависимости проекта командой `npm i`. Это установит все необходимые JavaScript-пакеты, включая TailwindCSS и другие инструменты для сборки (рис. 4.7).

```
npm i
```

Рис. 4.7: Установка npm-зависимостей

4.5 Локальный запуск сайта

Запускаю локальный сервер разработки командой `hugo server`. Hugo компилирует сайт и запускает веб-сервер на localhost:1313 (рис. 4.8).

```
hugo server
```

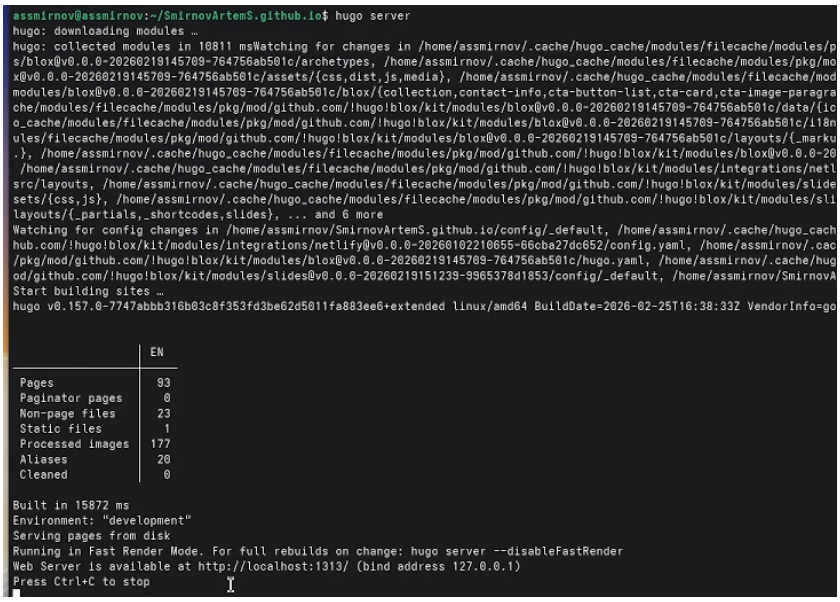

Рис. 4.8: Запуск локального сервера Hugo

Открываю браузер и перехожу по адресу `http://localhost:1313`. Сайт загружается корректно — отображается шаблон с демонстрационным контентом (рис. 4.9).

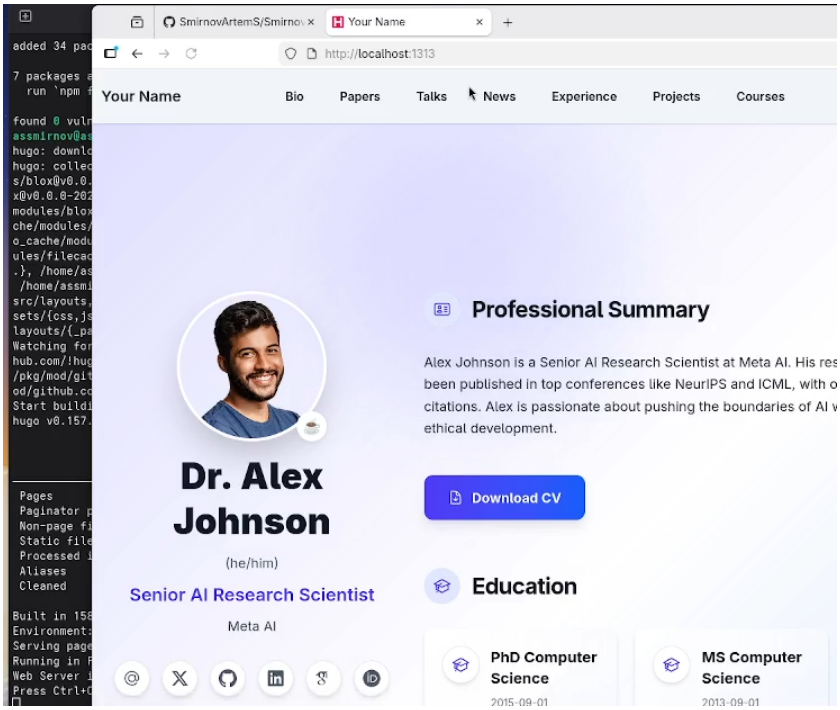

Рис. 4.9: Локальный сайт в браузере

4.6 Настройка baseURL

Открываю файл конфигурации `config/_default/config.yaml` и устанавливаю корректный baseURL, соответствующий адресу моего сайта на GitHub Pages (рис. 4.10).

```
nano config/_default/config.yaml
```

Устанавливаю значение:

```
baseURL: 'https://SmirnovArtemS.github.io/'
```

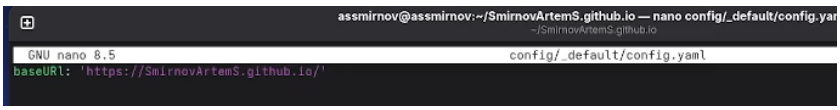

Рис. 4.10: Настройка baseURL в config.yaml

4.7 Настройка GitHub Pages

В настройках репозитория на GitHub перехожу в раздел Settings → Pages. В качестве источника (Source) выбираю GitHub Actions — это позволит автоматически собирать и публиковать сайт при каждом пуше в репозиторий (рис. 4.11).

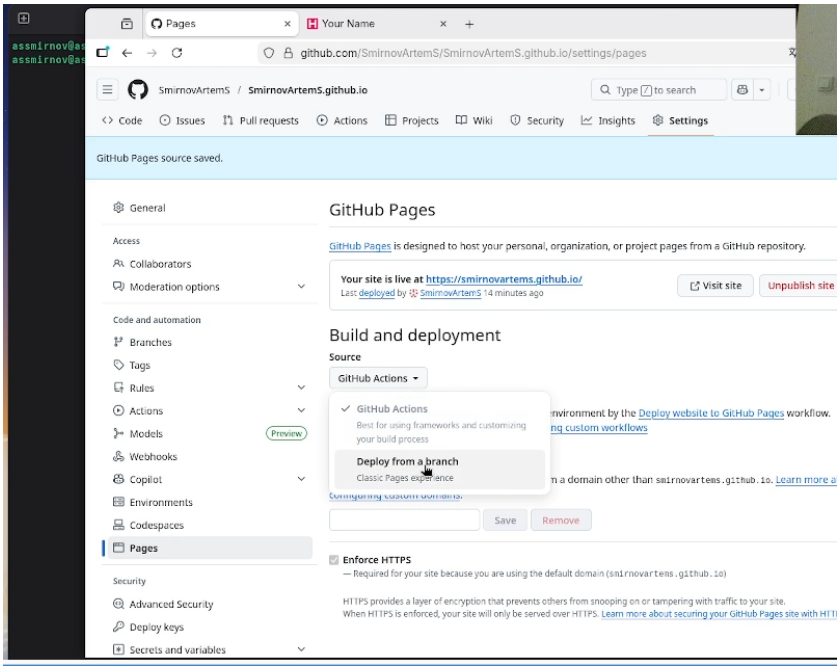

Рис. 4.11: Настройка GitHub Pages

4.8 Публикация сайта

Фиксирую все изменения и отправляю их на GitHub (рис. 4.12).

```
git add .
git commit -m "initial site setup"
```

```
git push
```

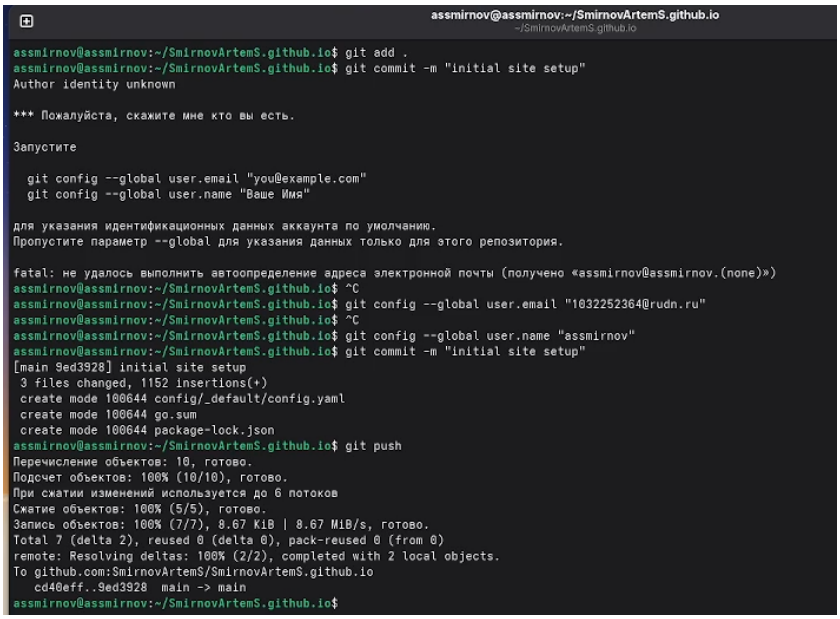

Рис. 4.12: Коммит и пуш изменений

Перехожу во вкладку Actions на GitHub и дожидаюсь успешного завершения workflow. Все три этапа (config, build, deploy) выполнены успешно (рис. 4.13).

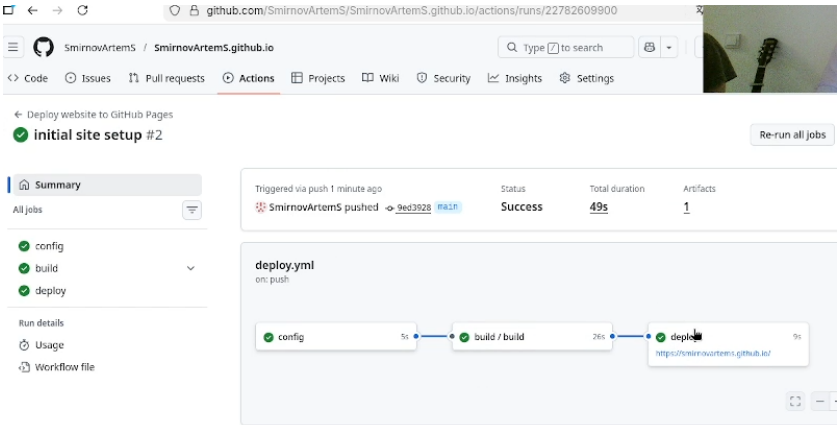

Рис. 4.13: Успешный деплой в GitHub Actions

4.9 Проверка результата

Открываю сайт по публичному адресу `https://SmirnovArtemS.github.io/`. Сайт отображается корректно — загружаются стили, шрифты и демонстрационный контент шаблона (рис. 4.14).

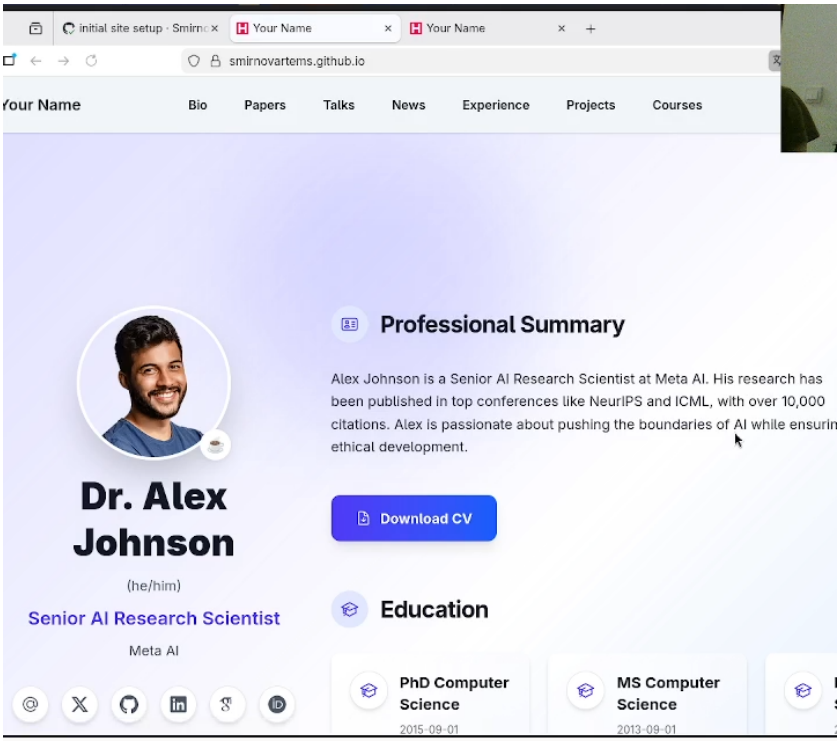

Рис. 4.14: Сайт на публичном адресе

5 Выводы

В ходе выполнения первого этапа индивидуального проекта установил необходимое программное обеспечение (Git, Go, Node.js, Hugo Extended), создал репозиторий на GitHub на основе шаблона Hugo Academic Theme, настроил baseURL и GitHub Pages для публикации сайта. Заготовка персонального сайта научного работника успешно размещена и доступна по адресу `https://SmirnovArtemS.github.io/`.

Список литературы